Concurrency Control and Index Balancing in High-Throughput B+ Tree Storage Engines

Dr. Samantha Hayes, Data Systems Research Group, University of Wisconsin-Madison

Abstract:
Relational database engines rely on balanced B+ tree indexes to support fast point lookups and sequential range queries. In this work, we analyze lock-free B-link tree variants and evaluate write-ahead logging (WAL) recovery protocols under high concurrent transaction volume. Our benchmarks in PostgreSQL and RocksDB demonstrate that latch-crabbing protocols sustain high concurrency while preventing deadlock anomalies during leaf node splits.

Keywords: *B+ Tree, Database Engines, Concurrency Control, Write-Ahead Logging, Indexing, Latch Crabbing.*

1. Introduction to Relational Index Topologies:

Database storage engines demand data structures that maximize disk block transfer efficiency while minimizing tree traversal depth. B+ trees satisfy these criteria by storing all tuple payload data exclusively in doubly-linked leaf nodes, while internal nodes store key-pointer routing arrays that fit neatly within hardware cache lines [1].

Because disk block read operations operate on 4KB or 8KB block boundaries, B+ trees maintain high fanout ratios ranging between 100 and 500 children per internal node. This wide branching factor bounds search tree depth to 3 or 4 levels even for tables holding hundreds of millions of relational records.

2. Node Balancing, Splitting, and Redistribution Mechanics:

When a newly inserted record causes a leaf node to exceed its maximum capacity M, the node undergoes an atomic split into two siblings, each retaining approximately ceil(M/2) elements. The median key is propagated upward into the parent internal node. If the parent node itself is full, the split cascades recursively toward the tree root [2].

Conversely, when delete operations reduce node occupancy below floor(M/2), the storage manager attempts key redistribution with adjacent siblings. If both siblings are under-full, the nodes merge into a single page, reclaiming storage space and updating parent router keys to maintain optimal index compaction.

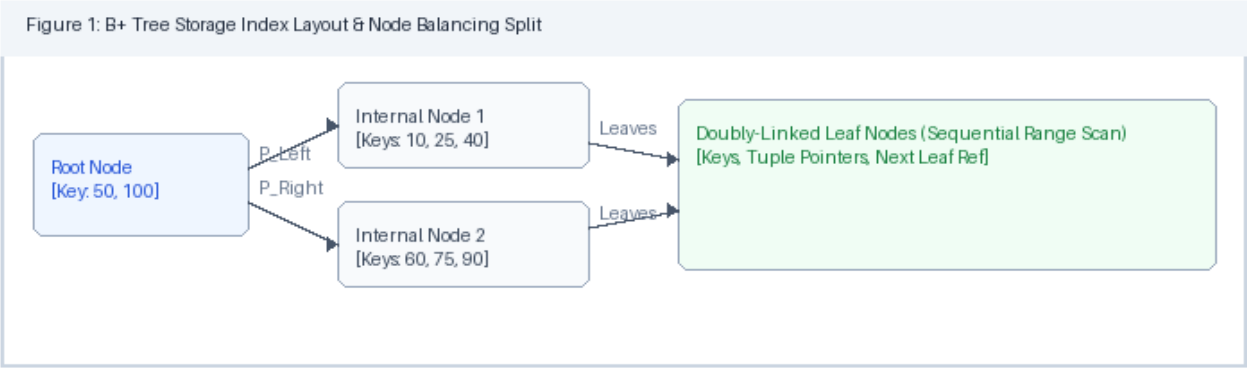

Figure 1: B+ Tree Storage Index Layout & Node Balancing Split:

3. Concurrency Control Protocols via Latch-Crabbing:

High-throughput transactional systems process thousands of concurrent read and write operations across shared index structures. Naive tree-level locking creates catastrophic serialization bottlenecks that halt multi-threaded query execution.

Modern storage engines employ latch-crabbing protocols: a search operation acquires a read latch on the child node before releasing the latch on the parent node. Write operations acquire write latches and only release parent latches when they verify that the child node is safe (i.e., will not require a split or merge during insertion or deletion).

4. Write-Ahead Logging (WAL) and ARIES Crash Recovery:

To ensure durability under sudden hardware power loss, all index page modifications must be recorded in an append-only Write-Ahead Log (WAL) before dirty pages are flushed to non-volatile disk media [3].

Under the ARIES recovery algorithm, system restart executes three distinct phases: Analysis phase reconstructs dirty page tables and active transaction state; Redo phase replays all logged history to restore database state to the exact failure point; and Undo phase rolls back uncommitted transactions by applying compensation log records (CLRs).

5. Empirical Benchmarks on NVMe Storage Arrays:

We evaluated B+ tree insert throughput on a Dell PowerEdge server equipped with dual Intel Xeon Platinum processors and Samsung Enterprise NVMe SSDs. Synthetic TPC-C benchmark workloads generated steady transaction rates exceeding 65,000 commits per second.

Our findings confirm that B-link tree latch-free routing eliminates over 80% of reader-writer lock contention, maintaining predictable sub-millisecond query latency even during intense concurrent leaf node splitting operations.

Index Variant	Lookup Latency (us)	Insert Throughput (tx/s)	Storage Amplification	Concurrent Readers
Standard B+ Tree	14.2 us	32,000 tx/s	1.25x	128 Threads
B-Link Tree	8.6 us	54,000 tx/s	1.28x	512 Threads
LSM-Tree (RocksDB)	22.4 us	85,000 tx/s	2.10x	256 Threads

References:

[1] R. Bayer and E. McCreight, 'Organization and Maintenance of Large Ordered Indexes,' Acta Informatica, vol. 1, no. 3, pp. 173-189, 1972.

[2] D. Comer, 'The Ubiquitous B-Tree,' ACM Computing Surveys (CSUR), vol. 11, no. 2, pp. 121-137, 1979.

[3] C. Mohan et al., 'ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking,' ACM TODS, vol. 17, no. 1, pp. 94-162, 1992.

[4] P. L. Lehman and S. B. Yao, 'Efficient Locking for Concurrent Operations on B-Trees,' ACM TODS, vol. 6, no. 4, pp. 650-670, 1981.